SYSTEM DESIGN EXERCISE

Library Check-in / Check-out System

Architecture, data model, AWS SaaS deployment, and reliability

Prepared as a system design proposal

1. Executive Summary

This proposal describes a library system that manages book titles and physical copies, tracks which member has borrowed each copy, records returns, and supports authorized staff adding new titles or copies. The design prioritizes correctness of circulation data, clear tenant isolation for a SaaS deployment, operational simplicity, and a migration path to greater scale.

The recommended starting point is a modular application deployed as a small number of independently scalable workloads, backed by a relational database. The domain is transaction-oriented: a physical copy must not be checked out to two members at the same time. A relational database with transactions and constraints is therefore the source of truth. Microservice boundaries are identified, but deploying every domain module as a separate service on day one would add operational overhead without a demonstrated need.

For a SaaS product, the design uses Amazon API Gateway or an Application Load Balancer, Amazon ECS on Fargate, Amazon Aurora PostgreSQL, Amazon SQS for asynchronous work, and managed security, observability, backup, and deployment services. Tenant identity is derived from authenticated claims, not trusted from an arbitrary client-supplied tenant ID.

2. Scope, Requirements, and Assumptions

2.1 Functional requirements

- Search and view book titles and their physical copies, including availability.
- Check out an available physical copy to an eligible library member.
- Return a copy currently checked out by that member and mark it available again.
- Add a book title and one or more physical copies; only authorized library staff can do this.
- View a member's active loans and basic loan history.
- Record who performed each circulation action and when it occurred.

2.2 Non-functional requirements

Concern	Target / design response
Correctness	A copy can have at most one active loan. Enforce this in the database transaction and constraints, not only in application code.
Availability	Initial target: 99.9% monthly availability, subject to agreed operational scope and AWS region choice.
Performance	Illustrative target: p95 under 300 ms for common reads and under 500 ms for checkout/return at normal load, excluding client/network latency.
Security	TLS in transit, encryption at rest, least-privilege IAM, staff/member authorization, audit trail, and tenant isolation.
Scalability	Stateless application tasks scale horizontally; database capacity and connection limits are monitored and scaled deliberately.
Recoverability	Automated backups and point-in-time recovery; explicit RPO/RTO agreed with the business.
Maintainability	Clear domain modules, API contracts, automated tests, infrastructure as code, and backward-compatible database migrations.

2.3 Assumptions and constraints

- A book title is a bibliographic record (for example, ISBN/title/author); a copy is an individual physical item with its own barcode or inventory identifier. Circulation applies to copies, not titles.
- Each library organization is a tenant. A tenant may have one or more branches; a physical copy belongs to one branch at a time.
- Members and staff are authenticated. Identity management may be provided by an external identity provider or Amazon Cognito.
- The first version supports basic checkout and return. Reservations, fines, renewals, inter-branch transfers, RFID integration, and offline circulation are out of scope unless requested.
- Loan periods, maximum active loans, and eligibility rules are configurable per tenant. Example defaults are illustrative, not hard-coded business policy.

- The system is an online application; if offline circulation is required, conflict resolution and reconciliation would need a separate design.
- Traffic and data-volume estimates are not provided. Capacity must be validated with expected tenants, copies, daily transactions, and peak concurrency before production sizing.

3. Domain Model and Business Invariants

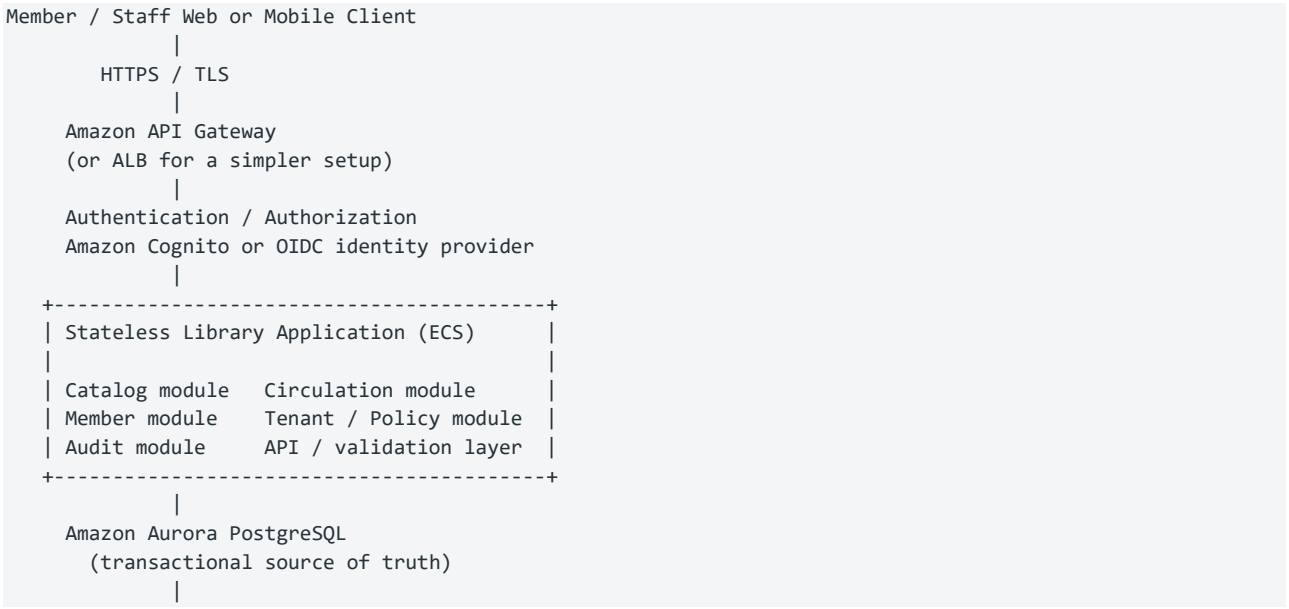
The key modeling decision is to separate a book title from its physical copies. One title can have many copies, each of which can be available, checked out, lost, damaged, or withdrawn.

Entity	Important fields	Purpose
Tenant	id, name, status, plan	SaaS customer / library organization.
Branch	id, tenant_id, name, address	Physical location within a tenant.
Member	id, tenant_id, external_subject, status	Person eligible to borrow books.
BookTitle	id, tenant_id, isbn, title, authors, publisher, metadata	Shared bibliographic description.
BookCopy	id, tenant_id, branch_id, title_id, barcode, status, version	Individually tracked physical item.
Loan	id, tenant_id, copy_id, member_id, checked_out_at, due_at, returned_at, status	Circulation record for one copy.
AuditEvent	id, tenant_id, actor_id, action, entity_type, entity_id, occurred_at, request_id	Append-oriented history for important actions.

3.1 Invariants

- A copy has no more than one active loan at any moment.
- Only an AVAILABLE copy can be checked out; only an actively checked-out copy can be returned.
- A checkout and its loan record are committed atomically.
- A return changes the active loan and copy status atomically.
- Every domain record is scoped to a tenant; reads and writes must not cross tenant boundaries.
- Only authorized staff can create or modify inventory; a member can act only on their own loans unless granted a staff role.
- A repeated request with the same idempotency key must not create a second loan or duplicate a return.

4. High-Level Architecture



```
      Transactional Outbox
             |
         Amazon SQS
             |
    Background worker(s) on ECS
      |            |            |
  Notifications  Analytics   Integrations
  (optional)     (optional)  (optional)

Cross-cutting: CloudWatch, X-Ray/OpenTelemetry, Secrets Manager,
KMS, WAF, CloudTrail, AWS Backup, CI/CD and infrastructure as code.
```

4.1 Service decomposition

Logical component	Responsibilities	Initial deployment
Catalog	Book titles, copies, search, inventory metadata.	Module in the main application.
Circulation	Checkout, return, loan rules, due dates, active-loan invariants.	Module in the main application; strongest transactional boundary.
Member & Access	Member status, staff roles, authorization mapping.	Module; identity authentication may be external.
Tenant & Policy	Tenant configuration, loan periods, limits, plan settings.	Module with strict tenant context.
Audit & Outbox	Durable audit records and events written in the same DB transaction as domain changes.	Module/table pattern; workers publish events asynchronously.
Notification / Integration worker	Email/SMS, reporting exports, third-party integrations.	Separate asynchronous worker when needed.

These are logical boundaries, not a requirement to deploy six microservices. For the first release, a modular monolith with clear module interfaces is likely the best trade-off: it reduces deployment and distributed-transaction complexity while preserving boundaries that can later be extracted. Split a module into an independently deployed service when there is a clear driver such as independent scaling, separate ownership, distinct availability requirements, or a different security boundary.

5. Physical Design and AWS SaaS Deployment

5.1 Production topology

```
AWS Region
  VPC across 3 Availability Zones
  ├── Public subnets
  │     └── Application Load Balancer / managed API entry
  ├── Private application subnets (AZ-a / AZ-b / AZ-c)
  │     ├── ECS Fargate tasks: API application
  │     └── ECS Fargate tasks: background workers
  └── Isolated data subnets
        └── Aurora PostgreSQL writer + reader(s), Multi-AZ

Supporting managed services:
  Route 53 | ACM | WAF | Cognito/OIDC | SQS | CloudWatch | X-Ray
  Secrets Manager | KMS | CloudTrail | AWS Backup | ECR
  CI/CD pipeline | Terraform / CloudFormation
```

5.2 AWS service choices

Capability	AWS choice	Reason / notes
DNS and TLS	Route 53 + AWS Certificate Manager	Custom domain routing and managed certificates.
API ingress	API Gateway or Application Load Balancer	API Gateway is useful for API throttling, quotas, and managed API features; ALB is a straightforward fit for containerized HTTP services. Choose one based on product needs.
Edge protection	AWS WAF; optionally CloudFront	Protect public endpoints with managed rules and rate limits; CloudFront is useful for static assets and global edge delivery.

Compute	Amazon ECS on Fargate	Runs containers without managing EC2 hosts; autoscaling and rolling deployments. EKS is justified only if Kubernetes capabilities are needed.
Relational database	Amazon Aurora PostgreSQL	Transactions, constraints, relational queries, managed backups, Multi-AZ resilience. RDS for PostgreSQL may be more cost-effective at modest scale.
Caching	ElastiCache for Redis, only if justified	Can cache hot catalog/search reads; never use cache as the authority for copy availability or checkout decisions.
Async messaging	Amazon SQS (+ DLQ)	Decouples non-critical side effects, retries transient failures, and buffers bursts.
Identity	Amazon Cognito or external OIDC provider	Authentication and token issuance; application still enforces tenant and resource authorization.
Secrets and encryption	Secrets Manager + AWS KMS	Managed secret rotation and encryption keys; avoid credentials in images or source control.
Logs and metrics	CloudWatch + X-Ray / OpenTelemetry	Central logs, service metrics, traces, dashboards, alarms, and request correlation.
Audit and AWS activity	Application audit table + CloudTrail	CloudTrail records AWS control-plane activity; it does not replace business-level audit events.
Backups	Aurora automated backups / PITR + AWS Backup as appropriate	Point-in-time recovery and retention policies; perform restore drills.
Container registry	Amazon ECR	Versioned, scanned container images.
Infrastructure / delivery	Terraform or CloudFormation + CI/CD	Repeatable environments, reviewed changes, automated tests, controlled deployments.

5.3 Multi-tenancy strategy

For an initial SaaS product, use a pooled model: shared application compute and shared Aurora PostgreSQL cluster, with tenant_id on every tenant-owned record. This keeps cost and operations manageable for many small and medium libraries. Enforce tenant scoping in the application data-access layer and consider PostgreSQL row-level security as defense in depth.

- Derive tenant identity from a validated token claim or trusted server-side mapping. Do not trust tenant_id from a request body as proof of access.
- Use composite unique constraints and foreign keys that include tenant_id where appropriate, reducing the risk of cross-tenant references.
- Include tenant_id in relevant indexes, audit records, metrics dimensions where cardinality is controlled, and background job payloads.
- Test tenant isolation explicitly, including object-ID guessing, exports, search, caches, logs, and asynchronous workers.
- For enterprise tenants with contractual isolation, offer a silo option (dedicated database or cluster, and potentially dedicated account) at higher cost.
- Avoid putting high-cardinality tenant identifiers into every CloudWatch metric dimension; use structured logs and targeted tenant-level diagnostics instead.

6. Logical Data Design

Illustrative relational schema (types simplified):

```
Tenant(
  id UUID PRIMARY KEY,
  name TEXT NOT NULL,
  status TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL
)

BookTitle(
  id UUID PRIMARY KEY,
  tenant_id UUID NOT NULL REFERENCES Tenant(id),
  isbn TEXT NULL,
  title TEXT NOT NULL,
  authors JSONB NULL,
```

```
  created_at TIMESTAMPTZ NOT NULL,
  UNIQUE (tenant_id, isbn)
)

BookCopy(
  id UUID PRIMARY KEY,
  tenant_id UUID NOT NULL,
  title_id UUID NOT NULL,
  branch_id UUID NOT NULL,
  barcode TEXT NOT NULL,
  status TEXT NOT NULL, -- AVAILABLE, CHECKED_OUT, LOST, DAMAGED, WITHDRAWN
  version BIGINT NOT NULL DEFAULT 0,
  UNIQUE (tenant_id, barcode),
  FOREIGN KEY (tenant_id, title_id) REFERENCES BookTitle(tenant_id, id)
)

Loan(
  id UUID PRIMARY KEY,
  tenant_id UUID NOT NULL,
  copy_id UUID NOT NULL,
  member_id UUID NOT NULL,
  checked_out_at TIMESTAMPTZ NOT NULL,
  due_at TIMESTAMPTZ NOT NULL,
  returned_at TIMESTAMPTZ NULL,
  status TEXT NOT NULL, -- ACTIVE, RETURNED
  FOREIGN KEY (tenant_id, copy_id) REFERENCES BookCopy(tenant_id, id)
)

-- PostgreSQL partial unique index: at most one active loan per copy
CREATE UNIQUE INDEX uq_active_loan_per_copy
ON Loan(tenant_id, copy_id)
WHERE status = 'ACTIVE';

-- Common access paths
CREATE INDEX ix_title_search ON BookTitle(tenant_id, title);
CREATE INDEX ix_copy_availability ON BookCopy(tenant_id, branch_id, status);
CREATE INDEX ix_member_active_loans ON Loan(tenant_id, member_id, status);
```

The exact schema would be refined for ISBN normalization, authors and subject indexing, branch rules, soft deletion, and migration strategy. A partial unique index is a useful database-level safety net; the application must still use transactions and return meaningful conflict responses.

7. API Design

Example REST endpoints. Authentication and tenant context are supplied by the access token and server-side authorization.

Method & endpoint	Purpose	Notes
GET /v1/books?query=...	Search titles and availability.	Paginated; availability may be summarized by branch.
POST /v1/books	Create a title and optionally initial copies.	Staff role required; validate ISBN and tenant-scoped uniqueness.
POST /v1/copies	Add physical copies to an existing title.	Staff role required; unique barcode per tenant.
POST /v1/copies/{copyId}/checkout	Check out a copy to the authenticated member or an authorized member.	Idempotency-Key recommended; server computes due date.
POST /v1/loans/{loanId}/return	Return an active loan.	Idempotency-Key recommended; duplicate return is handled safely.
GET /v1/members/{memberId}/loans?status=ACTIVE	List member loans.	Self access or staff permission; tenant-scoped.
GET /health/live	Process liveness.	No dependency checks that could cause

		restart storms.
GET /health/ready	Readiness for traffic.	Checks required dependencies with bounded timeouts.

7.1 Example responses and error semantics

- 201 Created: checkout succeeded; response includes loanId, copyId, memberId, checkedOutAt, dueAt, and status.
- 409 Conflict: copy is already checked out, or the requested state transition is no longer valid.
- 403 Forbidden: authenticated actor lacks permission for this tenant, member, branch, or operation.
- 404 Not Found: resource does not exist within the caller's tenant scope; avoid revealing another tenant's resource.
- 422 Unprocessable Entity: a business rule such as member eligibility or loan limit was not satisfied.
- 429 Too Many Requests: rate limit exceeded; include Retry-After when appropriate.
- 503 Service Unavailable: temporary dependency or capacity issue; clients may retry only when safe, preferably with the same idempotency key.

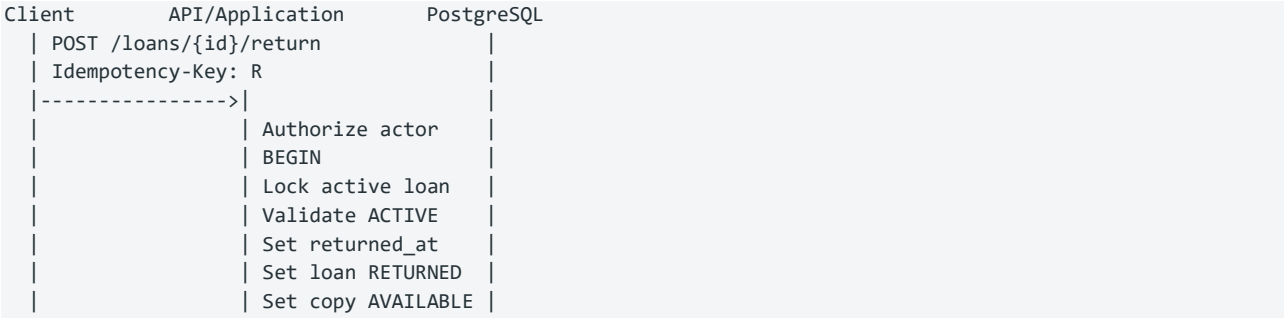
8. Key Sequences

8.1 Checkout a copy



Implementation notes: perform the copy state check and update in one database transaction. A row lock serializes concurrent attempts for the same copy; the unique active-loan index provides a second line of defense. Persist the idempotency key and response (or a durable operation record) so a retry after a network timeout returns the original result rather than creating another loan.

8.2 Return a copy



```
  |                 | INSERT AuditEvent  |
  |                 | INSERT OutboxEvent |
  |                 | COMMIT             |
  |                 |<-------------------|
  |<----------------| 200 + return result|
```

The loan status and copy status must change atomically. If the loan is already returned, the API can return the prior successful result for the same idempotency key; for a different request, return a documented conflict or an idempotent success according to the API contract. Choose one behavior consistently and test it.

8.3 Add a book and copies

```
Staff Client    API/Application       PostgreSQL
    | POST /books                         |
    |------------------>|                 |
    |                   | Authenticate /  |
    |                   | authorize STAFF |
    |                   | Validate fields |
    |                   | BEGIN           |
    |                   | Insert title    |
    |                   | Insert copies   |
    |                   | Insert audit    |
    |                   | Insert outbox   |
    |                   | COMMIT          |
    |                   |<----------------|
    |<------------------| 201 + IDs       |
```

If title creation and copy creation are submitted as one operation, they should commit together. If a request contains many copies, enforce a reasonable batch limit. For large imports, use an asynchronous import job with validation results and a downloadable error report.

9. Consistency, Concurrency, and Idempotency

Checkout and return are correctness-sensitive commands. Use the relational database as the authoritative state and do not rely on a cache, search index, or eventually consistent read replica to decide whether a copy can be borrowed.

- Use a short transaction and SELECT ... FOR UPDATE on the copy row (or an equivalent conditional update) to serialize competing checkouts.
- Keep a unique partial index for active loans per copy. Handle constraint violations as a normal domain conflict rather than a generic server error.
- Use an isolation level appropriate to the operation; READ COMMITTED plus row locking and constraints is sufficient for this invariant in a typical PostgreSQL design. Validate more complex cross-row policies separately.
- For member loan limits, concurrent checkouts of different copies can race. If strict enforcement is required, lock a member-level row or use a dedicated counter updated transactionally; do not rely on a non-atomic count query alone.
- Use idempotency keys for commands that may be retried after timeouts. Scope keys to tenant and actor, store request hash and outcome, and reject reuse with a different payload.
- Write audit and outbox records in the same transaction as the business change. Publish outbox events asynchronously; assume at-least-once delivery and make consumers idempotent.
- If catalog search later moves to OpenSearch, treat it as a derived read model. Search results can be slightly stale; checkout always revalidates against the primary database.

10. Mission-Critical Design Principles

Principle	Application to this system
Correctness first	Protect invariants with transactions, foreign keys, unique constraints, and explicit state transitions.
Failure isolation	Keep notification and analytics failures from blocking checkout/return. Use queues, timeouts, bounded retries, and dead-letter queues.

Graceful degradation	Catalog search or notifications may degrade temporarily; circulation writes should fail clearly rather than claim success without durable commit.
No silent data loss	A successful command response is returned only after the database commit. Use transactional outbox for downstream events.
Idempotent operations	Safe retries for checkout, return, imports, and event consumers.
Least privilege and zero trust	Authenticate every request, authorize every resource, use short-lived credentials, and validate tenant context end-to-end.
Observability by design	Structured logs, metrics, distributed traces, correlation IDs, and actionable alerts.
Automated recovery	Multi-AZ database, tested backups, infrastructure as code, documented recovery procedures, and regular restore drills.
Change safety	Backward-compatible schema changes, automated tests, canary/rolling deployments, and rollback or roll-forward plans.
Simplicity and clear ownership	Avoid premature microservices; define module contracts and extract services only when justified.
Capacity and overload control	Rate limits, connection pooling, queue depth alarms, autoscaling, and load tests.
Data governance	Retention, audit access, privacy controls, tenant deletion/export procedures, and documented data residency choices.

11. Reliability, Disaster Recovery, and Operations

11.1 Failure scenarios

Failure	Expected behavior / mitigation
Application task crashes	Load balancer removes unhealthy task; ECS replaces it. Requests may retry safely using idempotency keys.
Database writer unavailable	Multi-AZ failover; clients reconnect with bounded retries and jitter. Writes may be temporarily unavailable during failover.
SQS or worker outage	Core checkout/return remains available if DB transaction commits; outbox backlog grows and is alarmed. Publish resumes after recovery.
Notification provider fails	Retry asynchronously with backoff and DLQ; do not roll back a completed checkout.
Bad deployment	Health checks and deployment alarms stop rollout; roll back application image or roll forward with a fix. Database changes must remain backward-compatible.
Accidental data corruption	Restore to a point in time in an isolated environment, validate, then execute documented recovery steps.
Tenant isolation defect	Treat as a security incident: contain access, preserve logs, assess scope, notify according to obligations, remediate, and add regression tests.

11.2 Initial SLOs and recovery objectives

Proposed starting targets, to be agreed with the business: 99.9% monthly availability for core circulation APIs; p95 latency under 300 ms for reads and under 500 ms for checkout/return at expected load; RPO of 5 minutes or less and RTO of 60 minutes or less. These are targets, not guarantees, and require validation against the selected AWS configuration, backup settings, and operational maturity.

- Monitor request rate, error rate, latency percentiles, database connections/CPU/storage, lock waits, deadlocks, queue age/depth, DLQ messages, task restarts, and failed deployments.
- Alert on symptoms affecting users, not only infrastructure thresholds. Include synthetic checkout/return checks in a non-production or dedicated test tenant.
- Propagate a request/correlation ID across API, database audit metadata, outbox, and workers.
- Run load tests for peak concurrent checkouts and hot-copy contention; run chaos/failover exercises before claiming mission-critical readiness.
- Document runbooks for database failover, queue backlog, rollback, restoring data, key rotation, and tenant isolation incidents.

12. Security and Privacy

- Use OIDC/OAuth 2.0-compatible authentication and short-lived access tokens; enforce roles and resource-level authorization in the API.

- Treat tenant_id, member_id, and role claims as trusted only after signature, issuer, audience, and expiry validation. Map external identities to internal records server-side.
- Encrypt traffic with TLS and data at rest using AWS-managed or customer-managed KMS keys according to requirements.
- Place the database in private subnets with no public access. Restrict security groups and use IAM roles for workloads instead of long-lived AWS keys.
- Store secrets in Secrets Manager; rotate credentials and keys according to policy.
- Sanitize and validate inputs, use parameterized queries, set payload and batch limits, and rate-limit public endpoints.
- Minimize personal data in logs. Restrict access to loan history and define retention/deletion policies in line with applicable privacy requirements.
- Keep application audit events separate from infrastructure audit logs. Audit staff inventory changes, overrides, and circulation actions.

13. Testing Strategy

- Unit tests: loan eligibility, due-date calculation, copy/loan state transitions, and tenant policy rules.
- Integration tests against PostgreSQL: transaction rollback, partial unique index, foreign keys, concurrent checkout attempts, and idempotency behavior.
- API contract tests: status codes, validation errors, authentication, authorization, and backwards compatibility.
- Security tests: cross-tenant access attempts, privilege escalation, IDOR/object-ID guessing, injection, and secret exposure.
- Load tests: read-heavy catalog traffic, concurrent checkout of the same copy, bursts at opening time, and queue backlog recovery.
- Resilience tests: database failover, worker termination, SQS redrive/DLQ behavior, and deployment rollback.
- End-to-end tests: staff adds a title/copy, member checks it out, member returns it, and inventory availability is correct throughout.

14. Scaling and Evolution

Stage	Likely evolution	Trigger
Stage 1: MVP	Modular monolith, ECS Fargate, Aurora PostgreSQL, SQS for side effects.	Initial product and moderate traffic.
Stage 2: Growth	Tune indexes and queries, add read replicas for suitable read workloads, introduce caching for catalog metadata, add autoscaling and stronger tenant quotas.	Measured DB or application bottlenecks.
Stage 3: Larger catalog/search	Add OpenSearch or another dedicated search index as a derived read model; update it through outbox-driven events.	Advanced search needs or catalog query load beyond relational tuning.
Stage 4: Service extraction	Extract notifications/integrations or catalog into separately deployed services if independent scaling/ownership warrants it.	Organizational or scaling need, not a desire to maximize service count.
Stage 5: Enterprise isolation	Offer dedicated database/cluster or account for tenants with stricter isolation or residency needs.	Contractual, compliance, or noisy-neighbor requirements.

15. Key Trade-offs and Decisions

- Relational database over NoSQL for circulation: checkout/return depend on multi-entity atomic changes, constraints, and relational queries. A NoSQL store could work, but would require careful conditional writes and a different data model without an obvious benefit for this scope.
- Modular monolith before microservices: keeps the first release easier to operate and avoids distributed transactions. The design still defines boundaries that can be extracted later.
- Pooled multi-tenancy by default: lowers cost and operational overhead. Dedicated tenant infrastructure remains an option for enterprise requirements.

- SQS for side effects, not the transaction itself: queue-based asynchronous work improves resilience, but checkout must commit to the database before success is returned.
- Cache reads, not circulation truth: caches improve performance but may be stale. Final availability is always verified transactionally at checkout.
- Aurora versus standard RDS PostgreSQL: Aurora offers managed clustering and scaling options; RDS may be simpler and cheaper at low scale. Select based on measured requirements and cost.

16. Open Questions Before Production

- How many tenants, branches, titles, physical copies, active members, and daily circulation operations are expected?
- What availability, RPO, RTO, and support-hours commitments are required contractually?
- Are reservations, renewals, fines, loan limits, overdue handling, and inter-branch returns required?
- Can staff check out books on behalf of members? What overrides are permitted and audited?
- Are barcode scanners, RFID, self-service kiosks, or third-party library systems in scope?
- What are the data residency, retention, tenant deletion/export, and regulatory requirements?
- Does the SaaS need tenant-specific domains, branding, billing, quotas, or dedicated infrastructure?
- Is offline operation required during a network outage? If so, how are conflicting checkouts reconciled?

17. Conclusion

The design centers on a small set of explicit domain invariants and uses the database to enforce them. A modular application on AWS with Aurora PostgreSQL, stateless container workloads, and asynchronous workers is a pragmatic starting point for a multi-tenant SaaS product. Reliability comes from transactional state changes, idempotency, durable audit/outbox records, isolation of non-critical dependencies, tested recovery procedures, and strong tenant authorization—not from adding microservices by default.

The most important checkout guarantee is simple: one physical copy can have at most one active loan, even when two requests arrive concurrently or a client retries after a timeout. The architecture, data model, API contract, and operational controls are all designed around preserving that guarantee.